



Unidoc Finder

Motor de Pesquisa de Dissertações

David Sousa
Nuno Patrocínio

Supervisor: Nuno Leite

Relatório beta do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

30 de maio de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Unidoc Finder

Motor de Pesquisa de Dissertação

David Sousa, n.º48190, e-mail: a48190@alunos.isel.pt, tel.: 964861232

Nuno Patrocínio, n.º49442, e-mail: a49442@alunos.isel.pt, tel.: 965677039

Supervisor: Nuno Leite, e-mail: nuno.leite@isel.pt

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

30 de maio de 2026

Abstract

This report describes the development of *Unidoc Finder*, a web platform specialized in the search of theses and dissertations produced by Portuguese higher education institutions. The project arises from the need to centralize access to academic works that are distributed across different institutional repositories, making the search process simpler, faster, and more organized.

The proposed solution aggregates metadata automatically collected through a *web crawler*, without directly storing the original documents. For each work, information such as title, author, abstract, publication year, university, and link to the original repository is indexed. The system architecture is composed of a frontend developed in React and TypeScript, an API in Kotlin with Spring Boot, a PostgreSQL database for persistent storage, and Elasticsearch as the text search engine.

The system allows users to perform keyword searches, consult results ordered by relevance, apply filters, and directly access the original pages of the documents. The entire application is containerized with Docker, facilitating the execution and integration of the different components. In this way, *Unidoc Finder* aims to contribute to improving the visibility, reuse, and accessibility of national scientific production.

Resumo

O presente relatório descreve o desenvolvimento do *Unidoc Finder*, uma plataforma web especializada na pesquisa de teses e dissertações produzidas por instituições de ensino superior portuguesas. O projeto surge da necessidade de centralizar o acesso a trabalhos académicos que se encontram distribuídos por diferentes repositórios institucionais, tornando a pesquisa mais simples, rápida e organizada.

A solução proposta agrega metadados recolhidos automaticamente através de um *web crawler*, sem armazenar diretamente os documentos originais. Para cada trabalho são indexadas informações como título, autor, resumo, ano de publicação, universidade e ligação para o repositório de origem. A arquitetura do sistema é composta por um frontend desenvolvido em React e TypeScript, uma API em Kotlin com Spring Boot, uma base de dados PostgreSQL para armazenamento persistente e o Elasticsearch como indexador dos dados.

O sistema permite ao utilizador realizar pesquisas por palavras-chave, consultar resultados ordenados por relevância, aplicar filtros e aceder diretamente às páginas originais dos documentos. Toda a aplicação é containerizada com Docker, facilitando a execução e integração dos diferentes componentes. Desta forma, o *Unidoc Finder* pretende contribuir para melhorar a visibilidade, reutilização e acessibilidade da produção científica nacional.

Conteúdo

1	Introdução	1
1.1	Motivação	1
1.2	Objetivo do Projeto	1
1.3	Requisitos Funcionais	2
1.4	Arquitetura e Tecnologias Utilizadas	3
1.4.1	Aplicação Web (Frontend)	3
1.4.2	Backend	3
1.4.3	Motor de Pesquisa	3
1.4.4	Web Crawler	3
1.4.5	Infraestrutura	3
1.5	Diagrama	4
1.6	Desafios a Enfrentar	5
1.6.1	Extração de Dados	5
1.6.2	Relevância das Pesquisas	5
1.6.3	Velocidade de Pesquisa	5
1.6.4	Diferenças dos Filtros	5
2	Implementação	1
2.1	Frontend	1
2.1.1	Estrutura da aplicação	1
2.1.2	Componente principal e estado da interface	2
2.1.3	Comunicação com a API	2
2.1.4	Modelo de dados e mapeamento	2
2.1.5	Componentes da interface	3
2.1.6	Filtros e paginação	3
2.1.7	Estilização e responsividade	3
2.1.8	Execução em desenvolvimento e Docker	4
2.1.9	Limitações da implementação atual	4
2.2	API	5
2.2.1	Arquitetura da Aplicação	5
2.2.2	Tecnologias Utilizadas	5
2.2.3	Modelo de Dados	6
2.2.4	Endpoints Disponibilizados	6
2.2.5	Fluxo de Pesquisa	6
2.2.6	Comunicação com PostgreSQL e Elasticsearch	7
2.2.7	Containerização	7
2.2.8	Limitações da Implementação Atual	7
2.3	Elasticsearch	8
2.3.1	Motivação para a utilização do Elasticsearch	8
2.3.2	Modelo de Indexação	8
2.3.3	Processo de Indexação	9
2.3.4	Funcionamento das Pesquisas	9
2.3.5	Vantagens para o Projeto	10
2.4	Base de Dados	11
2.4.1	Modelo de Dados	11
2.4.2	Estrutura das Tabelas	11
2.4.3	Fluxo de Dados	11

2.5	Web Crawler	12
2.5.1	Repositórios configurados	12
2.5.2	Recolha HTTP	12
2.5.3	Metadados extraídos	13
2.5.4	Filtragem de teses e dissertações	14
2.5.5	Deduplicação	14
2.5.6	Construção do índice local	15
2.5.7	Parametrização da execução	15
2.5.8	Limitações da implementação atual	16
3	Conclusão	17
3.1	Tarefas Cumpridas	17
3.2	Desafios	17
3.3	Para o Futuro	18
	Referências	20

1 Introdução

Esta secção apresenta o projeto e explica a organização deste relatório.

1.1 Motivação

A motivação para este projeto surge da dificuldade sentida por estudantes, investigadores e outros interessados em encontrar teses académicas de forma centralizada e eficiente. Atualmente, muitas universidades disponibilizam a sua produção científica em repositórios próprios, mas estes encontram-se, na maioria dos casos, separados entre si e sem uma integração comum que permita uma pesquisa abrangente.

Esta fragmentação obriga os utilizadores a pesquisar manualmente em cada repositório universitário, tornando o processo demorado, repetitivo e pouco prático. Além disso, quando se recorre a motores de pesquisa generalistas, os resultados apresentados nem sempre são os mais relevantes ou específicos para o contexto académico, podendo dificultar o acesso a documentos úteis e credíveis.

Assim, torna-se relevante desenvolver uma solução que facilite a descoberta e consulta de trabalhos académicos produzidos em instituições de ensino superior. Ao melhorar o acesso a este conhecimento, pretende-se contribuir para a sua maior visibilidade, reutilização e valorização no contexto académico e científico.

1.2 Objetivo do Projeto

O principal objetivo deste projeto é desenvolver uma plataforma especializada na pesquisa de teses e dissertações de instituições de ensino superior em Portugal. Esta plataforma pretende agregar informação proveniente de diferentes repositórios institucionais, permitindo ao utilizador encontrar trabalhos académicos de forma mais simples, rápida e organizada.

Para suportar o seu funcionamento, o sistema será composto por um motor de pesquisa e um *web crawler*[1] responsável pela recolha automática de meta dados presentes nos repositórios públicos das universidades. Em vez de armazenar diretamente os documentos, a plataforma irá indexar apenas os meta dados relevantes e disponibilizar ligações que redirecionam o utilizador para o repositório original onde o trabalho se encontra disponível.

Com base nesta arquitetura, o projeto pretende atingir os seguintes objetivos:

- **Centralizar a pesquisa académica**, permitindo ao utilizador realizar uma única pesquisa que agregue resultados de vários repositórios universitários.
- **Indexar e organizar meta dados de dissertações**, incluindo título, autor, resumo, ano de publicação e ligação para o repositório original.
- **Melhorar a relevância dos resultados de pesquisa**, focando-se especificamente em trabalhos produzidos por instituições portuguesas e evitando resultados pouco relevantes.
- **Automatizar a recolha de informação**, através de um *web crawler* capaz de obter periodicamente novos dados dos repositórios institucionais.

Desta forma, o projeto pretende contribuir para facilitar o acesso à produção científica nacional e aumentar a visibilidade das dissertações desenvolvidas nas instituições de ensino superior portuguesas.

1.3 Requisitos Funcionais

Com base nos objetivos definidos, o sistema deverá cumprir um conjunto de requisitos funcionais que garantam a sua utilidade para os utilizadores finais. Estes requisitos descrevem as principais funcionalidades que a plataforma deverá disponibilizar.

- **Pesquisar teses e dissertações de faculdades portuguesas:** o sistema deverá permitir ao utilizador realizar pesquisas por palavras-chave, apresentando resultados relacionados com teses e dissertações provenientes de instituições de ensino superior em Portugal.
- **Disponibilizar links diretos para os repositórios originais:** cada resultado apresentado deverá incluir uma ligação para o repositório institucional original onde a tese ou dissertação se encontra disponível, garantindo o acesso à fonte oficial do documento.
- **Suportar filtros de pesquisa:** o sistema deverá permitir ao utilizador refinar os resultados através de filtros como ano de publicação, universidade, faculdade ou autor, facilitando a localização de trabalhos académicos específicos.

1.4 Arquitetura e Tecnologias Utilizadas

A arquitetura do sistema será composta pelos seguintes componentes principais:

1.4.1 Aplicação Web (Frontend)

A interface do utilizador será desenvolvida utilizando *React*[2], permitindo criar uma aplicação web moderna, responsiva e fácil de utilizar. Através desta interface, os utilizadores poderão pesquisar todo o tipo de documentos com foco em dissertações, visualizar resultados e aceder diretamente às páginas originais dos repositórios.

1.4.2 Backend

O *backend* será desenvolvido em *Kotlin*[3] utilizando o framework *Spring Boot*[4], sendo responsável pela lógica da aplicação, processamento das pesquisas e comunicação entre os diferentes componentes do sistema. O *backend* disponibilizará também uma *REST API* que permitirá ao *frontend* realizar consultas ao motor de pesquisa.

1.4.3 Motor de Pesquisa

Para a indexação e pesquisa da informação será utilizado o *Elasticsearch*[5][6], uma plataforma de pesquisa distribuída que permite realizar consultas rápidas e eficientes sobre grandes volumes de dados.

1.4.4 Web Crawler

Será desenvolvido um *web crawler* responsável por recolher automaticamente os meta dados dos documentos presentes nos repositórios institucionais.

1.4.5 Infraestrutura

Toda a aplicação está *containerizada* utilizando *Docker*, permitindo executar os diferentes componentes do sistema em ambientes isolados e facilitando o processo de *deploy* e manutenção da plataforma.

1.5 Diagrama

A seguinte figura representa a arquitetura do sistema.

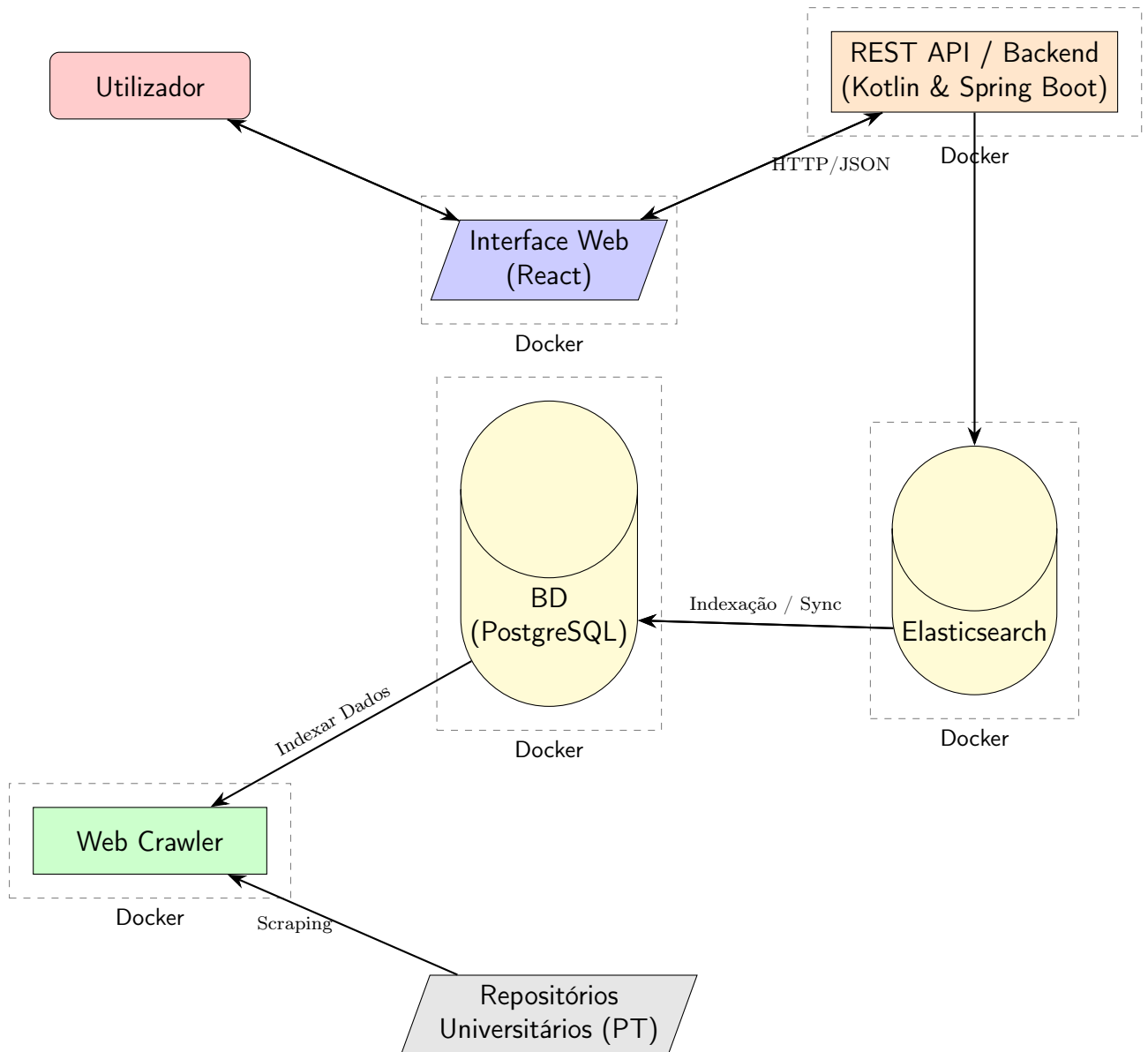


Figura 1: Arquitetura do sistema: interação entre utilizador, interface web, backend, bases de dados, crawler e os repositórios universitários.

1.6 Desafios a Enfrentar

Apesar das vantagens do sistema proposto, o desenvolvimento deste projeto apresenta alguns desafios que deverão ser considerados.

1.6.1 Extração de Dados

Os diferentes repositórios institucionais podem apresentar estruturas e formatos diferentes, o que pode dificultar a criação de um *crawler* capaz de extrair meta dados de forma consistente.

1.6.2 Relevância das Pesquisas

Configurar corretamente o sistema de indexação e os mecanismos de pesquisa será fundamental para garantir que os resultados apresentados aos utilizadores são relevantes e úteis.

1.6.3 Velocidade de Pesquisa

Garantir uma velocidade de pesquisa eficiente será um dos desafios do projeto, uma vez que o tempo de resposta pode ser influenciado por fatores como o volume de dados indexados e a complexidade das consultas realizadas.

1.6.4 Diferenças dos Filtros

Em cada repositório existem filtros diferentes. Será necessário analisar e filtrar a informação de cada repositório para identificar quais os filtros disponíveis e quais são comuns entre eles, de forma a integrá-los no motor de pesquisa.

Superar estes desafios será essencial para desenvolver uma plataforma eficiente e fiável que permita melhorar o acesso à produção académica das instituições de ensino superior portuguesas.

2 Implementação

Esta secção apresenta a implementação do projeto.

2.1 Frontend

O *frontend* foi implementado como uma aplicação web em *React* e *TypeScript*, utilizando *Vite* como ferramenta de desenvolvimento e empacotamento. Este componente é responsável pela interação direta com o utilizador, permitindo introduzir termos de pesquisa, seleccionar o tipo de documento, aplicar filtros e visualizar os resultados obtidos através da API do sistema.

A aplicação segue uma arquitetura de componentes. O ponto de entrada encontra-se em `src/main.tsx`, onde o componente principal **App** é montado no elemento **root** da página HTML. A partir desse componente são agregadas as principais áreas da interface: cabeçalho, barra de pesquisa, painel lateral de filtros e lista de resultados.

2.1.1 Estrutura da aplicação

O código-fonte encontra-se organizado por responsabilidade, separando a interface visual, a comunicação com a API, os tipos de dados e a transformação dos objetos recebidos do backend. A Tabela 1 apresenta os principais ficheiros e diretórios da implementação.

Ficheiro / Diretório	Responsabilidade
<code>src/App.tsx</code>	Componente principal da aplicação. Mantém o estado da pesquisa, filtros, tipo de documento selecionado e página atual.
<code>src/api/httpClient.ts</code>	Cliente HTTP genérico usado para comunicar com a API através de pedidos <code>fetch</code> . Também trata erros de resposta.
<code>src/api/searchApi.ts</code>	Função específica para pesquisar teses através do endpoint <code>/api/search</code> .
<code>src/hooks/useThesisSearch.ts</code>	<i>Hook</i> responsável por executar a pesquisa, controlar o estado de carregamento e guardar os resultados recebidos.
<code>src/mappers/thesisMapper.ts</code>	Conversão dos objetos recebidos da API para o modelo interno usado pelos componentes visuais.
<code>src/types</code>	Definição dos tipos <code>ThesisApiDto</code> , <code>UniversityApiDto</code> , <code>DocumentRecord</code> e <code>SearchFilters</code> .
<code>src/components</code>	Componentes reutilizáveis da interface, como barra de pesquisa, filtros, cartões de resultado e paginação.
<code>src/main/docker</code>	Ficheiros de suporte para execução do frontend em Docker durante o desenvolvimento.

Tabela 1: Organização principal do frontend.

Esta separação facilita a manutenção do código, uma vez que alterações na interface, no formato dos dados ou na comunicação com a API podem ser feitas em zonas bem delimitadas da aplicação.

2.1.2 Componente principal e estado da interface

O componente `App` centraliza o estado necessário para controlar a pesquisa. A aplicação guarda o tipo de documento selecionado, o texto escrito pelo utilizador, o termo de pesquisa submetido, os filtros ativos e a página atual. Foi definida uma constante `PAGE_SIZE` com o valor 10, fazendo com que cada pedido solicite, no máximo, dez resultados por página.

A distinção entre `queryInput` e `submittedQuery` permite que o utilizador escreva livremente na barra de pesquisa sem executar pedidos à API a cada alteração do campo. A pesquisa só é enviada quando o formulário é submetido. Nesse momento, o termo escrito passa para `submittedQuery` e a paginação volta à primeira página.

Depois de receber os documentos da API, o componente aplica filtros locais através da função `matchesSearch`. Esta função verifica se cada documento corresponde ao tipo selecionado, à universidade indicada, à categoria escolhida, às áreas introduzidas e ao intervalo de anos selecionado. Desta forma, a interface consegue refinar os resultados apresentados ao utilizador sem alterar imediatamente a estrutura da API.

2.1.3 Comunicação com a API

A comunicação com o backend está concentrada na pasta `src/api`. O ficheiro `httpClient.ts` define a função genérica `apiRequest`, que acrescenta o prefixo `/api` ao caminho recebido, envia os cabeçalhos necessários e interpreta a resposta de acordo com o respetivo tipo de conteúdo. Quando a resposta HTTP indica erro, é lançada uma exceção `ApiError`, guardando o código de estado e o conteúdo devolvido pelo servidor.

A pesquisa de documentos é feita no ficheiro `searchApi.ts`, através da função `searchThesis`. Esta função constrói os parâmetros `query`, `page` e `size` com `URLSearchParams` e executa um pedido para o endpoint:

```
/api/search?query=<termo>&page=<pagina>&size=<tamanho>
```

O *hook* `useThesisSearch` encapsula esta chamada. Quando existe um termo de pesquisa válido, o *hook* ativa o estado de carregamento, chama a API, converte os resultados recebidos com `mapThesisToDocument` e disponibiliza os documentos ao componente principal. Caso ocorra uma falha, a mensagem de erro é guardada e apresentada pela lista de resultados.

2.1.4 Modelo de dados e mapeamento

O frontend distingue entre o formato recebido do backend e o formato usado internamente pela interface. A API devolve objetos do tipo `ThesisApiDto`, contendo o identificador, título, resumo, ano, URL do documento e informação da universidade. Para apresentação na interface, estes objetos são transformados em `DocumentRecord`.

O mapeamento atribui o tipo `Thesis` e a categoria `Dissertation` aos documentos recebidos, guardando também o nome da universidade e o endereço do respetivo repositório. Esta camada de transformação evita que os componentes visuais dependam diretamente da estrutura exata da resposta do backend, tornando a interface mais flexível caso a API evolua.

2.1.5 Componentes da interface

A interface foi dividida em componentes reutilizáveis, cada um com uma responsabilidade específica:

- **Header**: apresenta o nome da aplicação, *Unidoc Finder*;
- **SearchBar**: contém a seleção do tipo de documento, o campo de pesquisa e o botão de submissão;
- **FilterSidebar**: apresenta os filtros por universidade, categoria, área e intervalo temporal;
- **FilterSection**: componente auxiliar usado para organizar visualmente cada grupo de filtros;
- **ResultsList**: apresenta os estados de carregamento, erro, ausência de resultados e a lista de documentos encontrados;
- **DocumentCard**: mostra cada resultado individual, incluindo ano, título, resumo, etiquetas, tipo de documento e universidade;
- **TagPill**: representa etiquetas visuais, sendo também usado para remover áreas selecionadas nos filtros;
- **UniversityBrand**: apresenta o nome da universidade associada ao documento.

Cada resultado apresentado inclui uma ligação externa para o URL original do documento. A ligação é aberta num novo separador, permitindo ao utilizador consultar o registo no repositório institucional sem perder o contexto da pesquisa na plataforma.

2.1.6 Filtros e paginação

O painel lateral permite refinar os resultados de acordo com vários critérios. O filtro de universidade é textual e compara o valor introduzido com o nome da instituição associada a cada documento. O filtro de categoria permite seleccionar a categoria **Dissertation**. O filtro de área permite introduzir termos manualmente e adicioná-los como etiquetas, sendo possível removê-los individualmente. O filtro de data disponibiliza intervalos predefinidos: 2020–2026, 2010–2019 e 2000–2009.

A paginação é controlada no componente **ResultsList**. O utilizador pode avançar ou recuar entre páginas através dos botões **Seguinte** e **Anterior**. O botão de página anterior fica desativado na primeira página, enquanto o botão de página seguinte depende da existência de uma página aparentemente completa, isto é, quando a quantidade de documentos recebidos é igual ao valor de `PAGE_SIZE`.

2.1.7 Estilização e responsividade

A estilização foi feita com CSS global e CSS Modules. O ficheiro `src/styles.css` define a estrutura geral da página, incluindo o fundo em gradiente, o espaçamento da aplicação e a grelha principal com duas colunas: uma para os filtros e outra para os resultados. Cada componente possui ainda o seu próprio ficheiro `.module.css`, o que reduz conflitos de estilos e mantém a apresentação visual associada ao respetivo componente.

A interface foi preparada para diferentes tamanhos de ecrã. Em ecrãs largos, os filtros surgem numa coluna lateral e os resultados ocupam a área principal. Em ecrãs mais pequenos, a grelha passa para uma única coluna, colocando os filtros acima dos resultados. A barra de pesquisa e os cartões de documentos também possuem regras específicas para se adaptarem a larguras reduzidas.

2.1.8 Execução em desenvolvimento e Docker

O projeto contém scripts `npm` para desenvolvimento, validação e geração da versão final da aplicação:

```
npm run dev
npm run build
npm run lint
npm run preview
```

O comando `npm run dev` inicia o servidor Vite com a opção `-host`, permitindo o acesso a partir do exterior do contentor Docker. O comando `npm run build` executa primeiro a validação TypeScript com `tsc -noEmit` e, em seguida, gera os ficheiros finais da aplicação com `vite build`.

A configuração do Vite define a porta 5173 para desenvolvimento e inclui um *proxy* para pedidos iniciados por `/api`. O destino do *proxy* pode ser configurado através da variável `VITE_BACKEND_URL`; quando esta variável não é definida, é usado o endereço `http://localhost:8080`. Esta configuração permite que o frontend comunique com o backend sem expor diretamente o endereço da API no código dos componentes.

A execução em Docker é suportada pelo ficheiro `Dockerfile.dev`, baseado na imagem `node:24.14.0-alpine`. O contentor instala as dependências com `npm install`, expõe a porta 5173 e executa o servidor de desenvolvimento com `npm run dev`. O ficheiro `docker-compose.yml` define o serviço `unidoc-frontend`, usando esse Dockerfile e atribuindo ao contentor o nome `unidoc-frontend`.

2.1.9 Limitações da implementação atual

Na implementação atual, os filtros são aplicados no cliente depois de a página de resultados ser recebida da API. Isto simplifica o frontend, mas significa que os filtros atuam apenas sobre os documentos presentes na página atual, e não necessariamente sobre todos os documentos existentes no índice. Uma evolução natural seria enviar os filtros juntamente com o pedido de pesquisa, permitindo que o backend aplicasse esses critérios diretamente sobre o conjunto total de resultados.

Além disso, a paginação baseia-se numa heurística simples: assume-se que existe uma próxima página quando o número de resultados recebidos é igual ao tamanho máximo da página. Uma alternativa mais completa seria a API devolver também o número total de resultados ou um indicador explícito de existência de próxima página. Por fim, o modelo `SearchFilters` já prevê um campo para autor, mas esse filtro ainda não se encontra refletido na interface atual.

2.2 API

A API constitui o elemento central da arquitetura do sistema, sendo responsável pela comunicação entre a interface web, a base de dados PostgreSQL e o motor de pesquisa Elasticsearch. Este componente foi desenvolvido em *Kotlin* utilizando o framework *Spring Boot*, disponibilizando um conjunto de endpoints REST que permitem efetuar pesquisas e gerir os dados armazenados na plataforma.

A utilização de uma API dedicada permite desacoplar o frontend dos mecanismos internos de armazenamento e pesquisa, facilitando a manutenção, evolução e escalabilidade da aplicação.

2.2.1 Arquitetura da Aplicação

O backend segue uma arquitetura em camadas, separando as responsabilidades de acesso aos dados, lógica de negócio e exposição dos serviços.

A estrutura principal da aplicação é composta pelos seguintes componentes:

- **Controllers:** recebem os pedidos HTTP provenientes do frontend e devolvem as respetivas respostas.
- **Services:** implementam a lógica de negócio da aplicação, coordenando o acesso às bases de dados e ao Elasticsearch.
- **Repositories:** responsáveis pela comunicação com o PostgreSQL e com o Elasticsearch.
- **Entities:** representam os dados armazenados na base de dados relacional.
- **DTOs (Data Transfer Objects):** utilizados para transportar informação entre o backend e o frontend de forma controlada.

Esta organização contribui para uma melhor separação de responsabilidades e facilita a manutenção do código à medida que o projeto evolui.

2.2.2 Tecnologias Utilizadas

A API foi desenvolvida recorrendo às seguintes tecnologias:

- Kotlin;
- Spring Boot;
- Spring Web MVC;
- Spring Data JPA;
- Spring Data Elasticsearch;
- PostgreSQL;
- Docker.

O Spring Boot foi escolhido devido à sua integração com o ecossistema Spring, simplificando a criação de APIs REST e a ligação a bases de dados relacionais e motores de pesquisa.

2.2.3 Modelo de Dados

A API trabalha essencialmente com duas entidades principais:

- **University:** representa uma instituição de ensino superior e contém informação como o nome da universidade e o endereço do respetivo repositório.
- **Thesis:** representa uma tese ou dissertação indexada pelo sistema, contendo o título, resumo, ano de publicação, URL do documento e a universidade associada.

A relação existente entre estas entidades é do tipo um-para-muitos, uma vez que uma universidade pode possuir várias dissertações.

2.2.4 Endpoints Disponibilizados

Atualmente, a API disponibiliza um conjunto de endpoints para gestão e pesquisa de dados.

Método	Endpoint	Descrição
GET	/api/search	Pesquisa dissertações
POST	/api/thesis	Regista uma dissertação
POST	/api/universities	Regista uma universidade
POST	/api/search/sync	Sincroniza o elastic

Tabela 2: Principais endpoints disponibilizados pela API.

Todos os pedidos e respostas são efetuados utilizando o formato JSON.

2.2.5 Fluxo de Pesquisa

Quando um utilizador realiza uma pesquisa através da interface web, o processo executado pela API segue os seguintes passos:

1. O frontend envia um pedido HTTP para o endpoint de pesquisa.
2. O controller recebe o pedido e valida os parâmetros recebidos.
3. O serviço de pesquisa executa uma consulta ao Elasticsearch.
4. O Elasticsearch devolve os documentos mais relevantes para o termo pesquisado.
5. Os resultados são convertidos para DTOs de resposta.
6. A API devolve os resultados ao frontend em formato JSON.

Este mecanismo permite obter tempos de resposta reduzidos mesmo quando o volume de documentos indexados aumenta significativamente.

2.2.6 Comunicação com PostgreSQL e Elasticsearch

A API utiliza simultaneamente duas tecnologias de armazenamento com objetivos distintos.

O PostgreSQL é responsável pelo armazenamento persistente dos dados da aplicação, garantindo consistência e integridade relacional.

O Elasticsearch é utilizado exclusivamente para pesquisa e indexação de documentos, permitindo efetuar consultas textuais rápidas e ordenar os resultados por relevância.

Esta abordagem combina as vantagens de uma base de dados relacional com as capacidades avançadas de um motor de pesquisa especializado.

2.2.7 Containerização

A API é executada dentro de um contentor Docker, permitindo uma configuração consistente entre ambientes de desenvolvimento e produção.

A utilização de Docker simplifica a distribuição da aplicação e facilita a integração com os restantes componentes do sistema, nomeadamente o PostgreSQL, o Elasticsearch e o frontend React.

2.2.8 Limitações da Implementação Atual

Na versão atual do projeto, a API encontra-se focada nas funcionalidades essenciais de pesquisa e gestão de dados.

Apesar destas limitações, a arquitetura implementada fornece uma base sólida para a evolução futura da plataforma.

2.3 Elasticsearch

O Elasticsearch é o componente responsável pela indexação e pesquisa dos metadados recolhidos dos repositórios universitários. Embora a informação seja armazenada de forma persistente na base de dados PostgreSQL, as pesquisas realizadas pelos utilizadores são efetuadas sobre índices mantidos no Elasticsearch.

Esta separação permite tirar partido das características específicas de cada tecnologia. O PostgreSQL é utilizado para garantir a persistência, integridade e consistência dos dados, enquanto o Elasticsearch é otimizado para pesquisas textuais rápidas sobre grandes volumes de informação.

2.3.1 Motivação para a utilização do Elasticsearch

Uma das decisões arquiteturais mais relevantes do projeto foi a utilização do Elasticsearch como motor de pesquisa dedicado. Embora fosse possível realizar pesquisas diretamente sobre a base de dados PostgreSQL, esta abordagem apresenta limitações quando o volume de dados aumenta e quando são necessárias funcionalidades avançadas de pesquisa textual.

O Elasticsearch foi escolhido pelas seguintes razões:

- **Pesquisa Full-Text:** permite pesquisar palavras e expressões em títulos, resumos e outros campos textuais de forma muito mais eficiente do que consultas SQL tradicionais.
- **Ordenação por relevância:** os resultados são classificados automaticamente de acordo com a sua proximidade ao termo pesquisado, apresentando primeiro os documentos mais relevantes.
- **Elevado desempenho:** foi concebido especificamente para operações de pesquisa e indexação, conseguindo responder rapidamente mesmo com centenas de milhares de documentos indexados.
- **Escalabilidade:** suporta crescimento horizontal através da distribuição dos índices por múltiplos nós, permitindo acomodar um aumento significativo do volume de informação.
- **Suporte para filtros:** possibilita combinar pesquisas textuais com filtros por universidade, autor, ano de publicação ou categoria sem degradação significativa do desempenho.

Foram consideradas outras alternativas, nomeadamente a utilização exclusiva do PostgreSQL ou de motores de pesquisa como Apache Solr. No entanto, o Elasticsearch apresentou uma melhor integração com o ecossistema Spring Boot através do módulo Spring Data Elasticsearch, além de disponibilizar uma API moderna e uma configuração relativamente simples para o contexto deste projeto.

2.3.2 Modelo de Indexação

Cada dissertação é representada no Elasticsearch através de um documento contendo os metadados mais relevantes para pesquisa.

A estrutura simplificada de um documento indexado é apresentada de seguida:

```
{
  "id": "...",
  "title": "Título da dissertação",
  "abstract": "Resumo da dissertação",
  "year": 2024,
  "url": "https://...",
  "authors": "...",
  "subjects": "...",
  "type": "...",
  "language": "...",
  "fileUrl": "https://...",
  "university": "Instituto Superior de Engenharia de Lisboa"
}
```

Os campos `title` e `abstract` constituem os principais elementos utilizados durante as pesquisas, sendo analisados e indexados pelo Elasticsearch para permitir pesquisas textuais eficientes.

2.3.3 Processo de Indexação

O processo de indexação inicia-se após a recolha dos metadados pelo *web crawler*. Os documentos obtidos são armazenados na base de dados PostgreSQL e posteriormente sincronizados com o Elasticsearch.

A Figura 1 ilustra este fluxo de informação.

O processo pode ser resumido da seguinte forma:

1. O *web crawler* recolhe os metadados dos repositórios universitários.
2. Os dados são armazenados na base de dados PostgreSQL.
3. O backend converte os registos armazenados em documentos de pesquisa.
4. Os documentos são indexados no Elasticsearch.
5. As pesquisas efetuadas pelos utilizadores são executadas sobre os índices criados.

Esta abordagem permite manter uma separação clara entre armazenamento persistente e otimização de pesquisa.

2.3.4 Funcionamento das Pesquisas

Quando um utilizador submete uma pesquisa através da interface web, o pedido é encaminhado para a API desenvolvida em Spring Boot. O backend comunica com o Elasticsearch e executa uma pesquisa sobre os campos indexados.

A pesquisa é realizada utilizando consultas *multi-match*, permitindo procurar simultaneamente em vários campos do documento, nomeadamente o título e o resumo da dissertação.

Após a execução da consulta, o Elasticsearch devolve os documentos ordenados por relevância. O backend transforma estes resultados em objetos de resposta e envia-os para o frontend, onde são apresentados ao utilizador.

2.3.5 Vantagens para o Projeto

A utilização do Elasticsearch permite que o sistema se aproxime do comportamento de um motor de pesquisa especializado, oferecendo uma experiência mais rápida e eficiente aos utilizadores.

Entre os principais benefícios destacam-se:

- Redução do tempo de resposta das pesquisas;
- Melhor relevância dos resultados apresentados;
- Capacidade de suportar grandes volumes de dissertações;
- Facilidade de implementação de novos filtros;
- Possibilidade de adicionar funcionalidades avançadas no futuro, como pesquisa aproximada (*fuzzy search*), sugestões automáticas e autocompletação.

Desta forma, o Elasticsearch constitui um elemento fundamental da arquitetura do Unidoc Finder, garantindo um mecanismo de pesquisa eficiente e escalável para a consulta de dissertações académicas provenientes de múltiplos repositórios universitários.

2.4 Base de Dados

A base de dados é responsável pelo armazenamento permanente da informação recolhida pelos repositórios universitários. Para este projeto foi escolhido o PostgreSQL, uma vez que oferece fiabilidade, facilidade de integração com o Spring Boot e suporte para relações entre entidades.

Embora as pesquisas sejam efetuadas através do Elasticsearch, todos os dados são inicialmente armazenados na base de dados. Desta forma, o PostgreSQL funciona como fonte principal de informação, enquanto o Elasticsearch é utilizado apenas para otimizar as pesquisas.

2.4.1 Modelo de Dados

A versão atual do sistema utiliza duas entidades principais:

- **University** – representa uma universidade e o respetivo repositório institucional.
- **Thesis** – representa uma tese ou dissertação indexada pelo sistema.

Cada universidade pode possuir várias teses associadas, existindo assim uma relação do tipo um-para-muitos entre as duas entidades.

2.4.2 Estrutura das Tabelas

A tabela `university` armazena o nome da instituição e o endereço do respetivo repositório.

A tabela `thesis` armazena informação sobre cada dissertação, incluindo o título, resumo, ano de publicação e ligação para o documento original.

2.4.3 Fluxo de Dados

O processo de armazenamento funciona da seguinte forma:

1. O *web crawler* recolhe os metadados dos repositórios universitários.
2. Os dados são guardados no PostgreSQL.
3. O Elasticsearch indexa a informação armazenada.
4. As pesquisas realizadas pelos utilizadores são efetuadas sobre os índices do Elasticsearch.

Esta abordagem permite combinar a fiabilidade de uma base de dados relacional com a rapidez de um motor de pesquisa especializado.

2.5 Web Crawler

O componente de recolha foi implementado como uma aplicação em JavaScript/Node.js. A execução principal encontra-se no ficheiro `build-index.js`, que coordena o processo de recolha, filtragem, deduplicação e envio destes dados para a API. A comunicação HTTP é isolada no ficheiro `fetcher.js`, a interpretação dos metadados OAI-PMH[7] é feita em `parser.js`, e a lista de repositórios a consultar encontra-se em `repositories.js`.

A implementação segue uma abordagem de *harvesting* através de OAI-PMH. Esta escolha reduz a dependência da estrutura visual de cada página HTML, porque os repositórios expõem os seus registos através de respostas XML normalizadas. Para cada repositório é construída uma chamada `ListRecords` com `metadataPrefix=oai_dc`. Quando a resposta contém um `resumptionToken`, o crawler usa esse token para obter a página seguinte de resultados, repetindo o processo até terminar a coleção ou até serem atingidos limites configurados.

2.5.1 Repositórios configurados

O ficheiro `repositories.js` define os repositórios institucionais a utilizar. Cada entrada contém um identificador, o nome da instituição e o endereço do respetivo endpoint OAI-PMH. A Tabela 3 apresenta os repositórios incluídos na configuração atual.

Repositório / Instituição
Instituto Politécnico de Lisboa
Universidade de Lisboa
Universidade Nova de Lisboa
Universidade do Minho
Universidade do Porto
Universidade de Coimbra
Universidade de Aveiro
Universidade da Beira Interior
Universidade do Algarve
Universidade de Évora
Universidade de Trás-os-Montes e Alto Douro
Universidade dos Açores
Universidade da Madeira
Universidade Aberta
ISCTE – Instituto Universitário de Lisboa

Tabela 3: Repositórios configurados no crawler.

A execução pode ser limitada a um subconjunto destes repositórios através da variável de ambiente `REPOS`. Por exemplo, `REPOS=u1,run,ipl` permite executar o processo apenas para a Universidade de Lisboa, Universidade Nova de Lisboa e Instituto Politécnico de Lisboa.

2.5.2 Recolha HTTP

O ficheiro `fetcher.js` contém a função `fetchText`, responsável por descarregar as respostas XML dos endpoints OAI-PMH. Esta função define um *User-Agent* próprio, `UnidocFinderCrawler/0` e envia o cabeçalho `Accept` com preferência por XML. Para aumentar a robustez, cada

pedido possui um tempo limite configurável e pode ser repetido quando ocorre uma falha temporária.

Os principais parâmetros de recolha são configuráveis por variáveis de ambiente:

- `FETCH_TIMEOUT_MS`: tempo máximo de espera por pedido HTTP, com valor por defeito de 30000 ms;
- `FETCH_RETRIES`: número de tentativas adicionais após uma falha, com valor por defeito de 2;
- `USER_AGENT`: identificação enviada pelo crawler ao servidor;
- `CRAWL_DELAY_MS`: intervalo entre páginas OAI-PMH consecutivas, com valor por defeito de 250 ms.

Desta forma, o crawler evita terminar imediatamente perante erros pontuais de rede e reduz a probabilidade de sobrecarregar os servidores consultados.

2.5.3 Metadados extraídos

A extração dos metadados é feita a partir dos elementos Dublin Core presentes dentro de cada registo OAI-PMH. O parser ignora registos marcados como removidos e, para cada registo válido, extrai os campos relevantes para a pesquisa e apresentação dos resultados.

Um registo OAI-PMH em Dublin Core contém elementos do seguinte tipo:

```
<record>
  <header>
    <identifier>oai:repositorio.exemplo.pt:123456789/10</identifier>
    <datestamp>2024-05-01</datestamp>
  </header>
  <metadata>
    <oai_dc:dc>
      <dc:title>Título da dissertação</dc:title>
      <dc:creator>Nome do autor</dc:creator>
      <dc:date>2024</dc:date>
      <dc:description>Resumo do trabalho académico</dc:description>
      <dc:subject>Inteligência Artificial</dc:subject>
      <dc:type>masterThesis</dc:type>
      <dc:language>por</dc:language>
      <dc:identifier>https://repositorio.exemplo.pt/handle/...</dc:identifier>
    </oai_dc:dc>
  </metadata>
</record>
```

A estrutura interna guardada pelo crawler contém os seguintes campos:

- `repository`: identificador curto do repositório;
- `university`: nome da instituição;
- `title`: título do documento;

- **authors:** lista de autores;
- **description:** resumo ou descrição;
- **year:** ano de publicação, extraído dos campos de data ou, quando necessário, dos identificadores;
- **subjects:** palavras-chave ou áreas temáticas;
- **type:** tipo de documento declarado pelo repositório;
- **language:** idioma do documento;
- **url:** ligação para a página pública do registo no repositório;
- **fileUrl:** ligação direta para o PDF, quando existe;
- **oaiIdentifier:** identificador OAI-PMH do registo;
- **datestamp:** data de atualização indicada pelo repositório.

Durante esta fase também é feita limpeza textual: remoção de marcas XML/HTML, decodificação de entidades XML, normalização de espaços e eliminação de valores vazios. O objetivo é transformar a resposta XML original num objeto mais simples e adequado para indexação.

2.5.4 Filtragem de teses e dissertações

Nem todos os registos expostos por um repositório correspondem a teses ou dissertações. Por esse motivo, o crawler aplica uma função de classificação simples, `isThesis`, que procura termos característicos no título, resumo, tipo, assuntos, URL, PDF e identificador OAI-PMH.

O padrão de deteção inclui expressões como `masterthesis` e `doctoralthesis`. Esta abordagem permite abranger repositórios que usam valores normalizados, como `info:eu-repo/semantics/masterThesis`, e também repositórios que usam descrições textuais menos uniformes.

Além da filtragem automática por tipo de documento, existe uma filtragem opcional por texto. O filtro pode ser passado como argumento de linha de comandos ou através da variável `FILTER`. O crawler normaliza o texto, remove acentos, ignora maiúsculas/minúsculas e divide a expressão em termos relevantes, excluindo algumas *stopwords*. Um documento é mantido se a expressão completa ou todos os termos relevantes forem encontrados nos seus metadados principais.

2.5.5 Deduplicação

Para evitar armazenar várias vezes o mesmo documento, cada registo recebe uma chave estável criada a partir da melhor informação disponível. A prioridade usada é a seguinte:

1. URL da página do repositório;
2. URL direta do PDF;
3. identificador OAI-PMH;

4. combinação entre título, autores e ano.

A chave é convertida num hash SHA-1 e guardada em memória durante a execução. Quando o mesmo hash aparece novamente, o registo é considerado duplicado e não é escrito nos ficheiros finais.

2.5.6 Construção do índice local

Após a extração e filtragem, os documentos são escritos de forma incremental para o ficheiro `documents.jsonl`. Cada linha contém um documento em JSON, permitindo processar grandes volumes de dados sem manter todos os documentos completos em memória. Em paralelo, o crawler guarda os offsets de cada linha no ficheiro `documents-offsets.json`, permitindo carregar diretamente apenas os documentos necessários durante uma pesquisa.

O índice invertido é construído com base nos metadados normalizados de cada documento. A função `addDocumentToIndex` adiciona termos ao índice com pesos diferentes consoante o campo onde aparecem. Os pesos aplicados são:

Campo	Peso
Título	9
Assuntos / palavras-chave	5
Autores	4
Ano	3
Tipo de documento	2
Universidade	2
Resumo	1

Tabela 4: Pesos utilizados na construção do índice invertido local.

Esta ponderação dá maior relevância a documentos cujo termo pesquisado aparece no título ou nas palavras-chave, sem ignorar outros campos úteis como autores, ano, universidade e resumo. Para controlar o tamanho do índice, apenas os termos mais relevantes de cada documento são mantidos, de acordo com o limite `MAX_TERMS_PER_DOCUMENT`.

No final da execução são gerados quatro ficheiros principais:

- `documents.jsonl`: documentos normalizados, um por linha;
- `documents-offsets.json`: posição de cada documento dentro do ficheiro JSONL;
- `inverted-index.json`: índice invertido compacto usado pela pesquisa local;
- `metadata.json`: informação sobre a geração do índice, incluindo repositórios usados, número de documentos e número de termos indexados.

2.5.7 Parametrização da execução

A execução do crawler pode ser ajustada sem alterar o código-fonte. Os parâmetros principais são:

- `REPOS`: lista de repositórios a processar;
- `MAX_RECORDS_PER_REPO`: número máximo de registos a processar por repositório;

- `MAX_DOCUMENTS`: número máximo global de documentos a guardar;
- `PROGRESS_EVERY`: intervalo de registos processados entre mensagens de progresso;
- `MAX_DESCRIPTION_CHARS`: tamanho máximo do resumo guardado em cada documento;
- `INDEX_DESCRIPTION_CHARS`: número de caracteres do resumo usados para indexação;
- `INDEX_DIR`: diretoria onde são escritos os ficheiros finais.

A construção do índice é executada com o comando:

```
npm run build:index
```

Também é possível executar o crawler com filtros ou limites, por exemplo:

```
REPOS=ul,run,ipl MAX_RECORDS_PER_REPO=1000 npm run build:index  
FILTER="inteligencia artificial" npm run build:index
```

2.5.8 Limitações da implementação atual

A implementação atual privilegia simplicidade e baixo consumo de memória, mas apresenta algumas limitações. Em primeiro lugar, o crawler recolhe apenas metadados disponibilizados por OAI-PMH, não extraindo o texto integral dos PDFs. Isto significa que a pesquisa incide sobretudo sobre título, autores, resumo, assuntos e restantes campos Dublin Core[8].

Em segundo lugar, a identificação de teses e dissertações depende de padrões textuais. Embora estes padrões cubram os casos mais comuns, podem existir registos mal classificados pelo repositório ou documentos cujo tipo esteja descrito de forma inesperada. Nesses casos, alguns documentos relevantes podem não ser incluídos ou alguns documentos menos relevantes podem passar o filtro.

Por fim, a interpretação XML é feita por expressões regulares simples. Esta solução é suficiente para respostas OAI-PMH previsíveis, mas uma implementação mais robusta poderia utilizar uma biblioteca XML dedicada, especialmente se fosse necessário suportar metadados com estruturas mais complexas ou namespaces menos consistentes.

3 Conclusão

Este capítulo resume o trabalho desenvolvido no âmbito do projeto Unidoc Finder, os principais resultados alcançados, os desafios encontrados durante a implementação e as melhorias que poderão ser realizadas em versões futuras. Importa referir que o projeto se encontra ainda numa versão beta, pelo que algumas funcionalidades foram implementadas apenas de forma inicial e outras permanecem incompletas ou sujeitas a evolução.

Apesar dessas limitações, foi possível construir uma base funcional para uma plataforma de pesquisa de dissertações, composta por vários componentes integrados entre si. O sistema desenvolvido permite demonstrar a viabilidade da solução proposta, especialmente no que diz respeito à recolha, organização, indexação e pesquisa de metadados provenientes de repositórios académicos portugueses.

3.1 Tarefas Cumpridas

Durante o desenvolvimento do projeto foi possível concretizar vários dos objetivos definidos inicialmente. Foi implementada uma aplicação web em React e TypeScript, permitindo ao utilizador introduzir termos de pesquisa, visualizar resultados e aplicar alguns filtros sobre os documentos apresentados. A interface foi organizada em componentes reutilizáveis, facilitando a manutenção e evolução futura da aplicação.

Foi também desenvolvida uma API em Kotlin com Spring Boot, responsável pela comunicação entre o frontend, a base de dados PostgreSQL e o Elasticsearch. Esta API disponibiliza endpoints para pesquisa, registo de dissertações, registo de universidades e sincronização com o motor de pesquisa. Desta forma, foi criada uma separação clara entre a interface do utilizador, a lógica da aplicação e os mecanismos de armazenamento e pesquisa.

Ao nível do armazenamento, foi configurada uma base de dados PostgreSQL para guardar de forma persistente informação sobre universidades e dissertações. Em paralelo, foi integrado o Elasticsearch como motor de pesquisa textual, permitindo indexar os metadados recolhidos e apresentar resultados ordenados por relevância.

Outro componente essencial desenvolvido foi o web crawler, responsável pela recolha automática de metadados a partir de diferentes repositórios institucionais. Este componente utiliza o protocolo OAI-PMH para obter registos em formato Dublin Core, extraíndo informação como título, autor, resumo, ano, tipo de documento, idioma, palavras-chave e ligações para os documentos originais. Foram ainda implementados mecanismos de filtragem e de deduplicação.

Por fim, os diferentes componentes da aplicação foram preparados para execução com Docker, o que simplifica a configuração do ambiente de desenvolvimento e facilita a integração entre frontend, backend, base de dados, Elasticsearch.

3.2 Desafios

O maior desafio encontrado ao longo do projeto esteve relacionado com a recolha de dados a partir de diferentes repositórios académicos. Embora vários repositórios disponibilizem metadados através de OAI-PMH, a forma como esses dados são preenchidos nem sempre é uniforme. Campos como o tipo de documento, o resumo, os autores, as palavras-chave ou o ano de publicação podem variar bastante entre instituições, tanto no formato como na qualidade da informação disponibilizada.

Esta falta de uniformização dificultou a criação de um processo de extração completamente consistente. Alguns repositórios utilizam valores normalizados para identificar teses e dissertações, enquanto outros recorrem a descrições textuais menos previsíveis. Como consequência, foi necessário implementar uma lógica de classificação baseada em padrões textuais, capaz de reconhecer termos associados a dissertações e teses. No entanto, esta abordagem não é perfeita e pode originar falsos positivos ou excluir documentos relevantes quando os metadados estão incompletos ou mal classificados.

Outro desafio importante foi lidar com a diversidade dos próprios repositórios. Cada instituição pode expor coleções, identificadores, ligações e filtros de forma diferente, o que torna difícil criar um crawler totalmente genérico. Mesmo recorrendo ao OAI-PMH, continuaram a existir diferenças na estrutura dos registos, na disponibilidade de campos e na existência ou não de ligações diretas para os ficheiros PDF.

A deduplicação dos documentos também representou uma dificuldade relevante. O mesmo trabalho pode aparecer com URLs diferentes, identificadores distintos ou pequenas variações no título e nos autores. Para minimizar este problema, foi criada uma chave estável baseada em vários campos, mas a eliminação completa de duplicados exigiria uma estratégia mais avançada de comparação semântica.

Ao nível da pesquisa, o desafio passou por garantir que os resultados apresentados fossem relevantes para o utilizador. A configuração do Elasticsearch, a escolha dos campos indexados e a ponderação entre título, resumo, autores, assuntos e universidade são fatores que influenciam diretamente a qualidade dos resultados. Nesta versão beta, a pesquisa já funciona sobre os metadados indexados, mas ainda existe margem para melhorar a ordenação por relevância e a combinação com filtros.

Também existiram limitações na integração entre frontend e backend. Atualmente, alguns filtros são aplicados no cliente depois de a página de resultados ser recebida, o que significa que estes não atuam necessariamente sobre todo o conjunto de documentos existentes. Esta solução permitiu avançar com uma interface funcional, mas deverá ser revista numa versão futura para que os filtros sejam processados diretamente pelo backend e pelo Elasticsearch.

3.3 Para o Futuro

Como trabalho futuro, uma das prioridades será tornar o processo de recolha de dados mais robusto e abrangente. Será importante adicionar suporte a mais repositórios académicos portugueses e melhorar a adaptação do crawler às diferenças existentes entre instituições. A utilização de uma biblioteca XML dedicada, em vez de expressões regulares simples, poderá também tornar a interpretação dos registos OAI-PMH mais fiável.

No backend, deverá ser implementado um sistema de filtros mais completo, permitindo que critérios como universidade, autor, ano, tipo de documento, idioma e área científica sejam enviados diretamente para a API e aplicados sobre o índice do Elasticsearch. Esta alteração tornará os resultados mais corretos e coerentes, sobretudo quando o volume de documentos indexados aumentar.

Também será relevante melhorar a paginação, fazendo com que a API devolva informação sobre o número total de resultados e a existência de páginas seguintes. Esta funcionalidade permitiria ao frontend apresentar uma navegação mais clara e informativa para o utilizador.

No motor de pesquisa, poderão ser exploradas funcionalidades mais avançadas do Elasticsearch, como pesquisa aproximada, autocompletação, sugestões de termos, nor-

malização linguística e maior ponderação dos campos mais relevantes. Estas melhorias contribuiriam para tornar a pesquisa mais flexível e útil em cenários reais.

Em conclusão, o Unidoc Finder encontra-se ainda numa fase beta e incompleta, mas já demonstra o potencial de uma plataforma centralizada para pesquisa de dissertações académicas. O trabalho desenvolvido permitiu validar a arquitetura proposta e identificar os principais obstáculos técnicos do projeto, sobretudo na recolha e normalização de dados provenientes de repositórios distintos. Com a continuação do desenvolvimento, o sistema poderá tornar-se uma ferramenta mais completa, fiável e útil para estudantes, investigadores e instituições de ensino superior.

Referências

- [1] Patel, J. M. (2021). *Getting structured data from the internet: Running web crawlers/scrapers on a big data production scale*. Apress.
- [2] React. Disponível em: <https://react.dev>. Acedido a: 07/03/2026.
- [3] Kotlin Lang - <https://kotlinlang.org>. Acedido a: 07/03/2026.
- [4] Spring Boot - <https://spring.io/projects/spring-boot>. Acedido a: 07/03/2026.
- [5] Elasticsearch - <https://www.elastic.co/docs>. Acedido a: 07/03/2026.
- [6] Gupta, M. (2019). *Mastering Elasticsearch 7.x: Build modern, fast, and scalable search applications*. Packt Publishing.
- [7] Open Archives Initiative Protocol for Metadata Harvesting - <https://www.openarchives.org/pmh/>. Acedido a 26/04/2026.
- [8] Dublin Core - <https://www.openarchives.org/OAI/openarchivesprotocol.html#dublincore>. Acedido a 26/04/2026.